Text Analysis

MSIS 521
Professor Hossein Ghasemkhani

FOSTER SCHOOL OF BUSINESS
UNIVERSITY OF WASHINGTON

1

Agenda

- Similarity between documents
- Clustering
  - K-means Clustering
  - Hierarchical Clustering
- LDA Topic Modeling
- Sentiment Analysis

FOSTER SCHOOL OF BUSINESS

2

2

Similarity and Clustering

3

3

How Similar Are These Documents?

- He is a nice guy
- The instructor is a nice guy
- The TA is really good
- I love my TA

4

4

How Similar Are These Documents?

- Mathematical Similarity

docs	t1	t2	t3
D1	1	0	1
D2	1	0	0
D3	0	1	1
D4	1	0	0
D5	1	1	1
D6	1	1	0
D7	0	1	0
D8	0	1	0
D9	0	0	1
D10	0	1	1

docs	t1	t2	t3
D1	2	0	3
D2	1	0	0
D3	0	4	7
D4	3	0	0
D5	1	6	3
D6	3	5	0
D7	0	8	0
D8	0	10	0
D9	0	0	1
D10	0	3	5

FOSTER SCHOOL OF BUSINESS

5

5

Setup

- D is a collection, which contains m documents

$$D=\{d_1, d_2,\ldots d_i\ldots d_m\}\ i=1,2\ldots m$$

- There are n words among m documents.

$$di=\{w_{i1},w_{i2},\ldots w_{ij},\ldots w_{in}\}\ i=1,2\ldots m\ ,\ \ j=1,2,\ldots.n$$

- Q is a benchmark document

$$Q= \{q_1,q_2,\ldots q_i,\ldots.q_n\}\ i=1,2,\ldots.n$$

- If similarity(q, d_i) > similarity(q, d_j), then document d_i is more relevant than d_j

FOSTER SCHOOL OF BUSINESS

6

6

Similarity Measures

- A **similarity measure** can represent the similarity between two **vectors**.
- A similarity measure is a **function** which computes the **degree of similarity** between a pair of vectors.
- Note: There are a large number of similarity measures proposed in the literature, and the measure we use depends on the purpose of analysis.

FOSTER SCHOOL OF BUSINESS

7

7

Classic Similarity Measures

- Similarity measures are mapped to the range [-1, 1] or [0, 1]
- 0 or -1 shows minimum similarity
- 1 shows maximum similarity
- Simple normalization through Min-Max Scaling can be used to achieve the range:
  - e.g.: $s' = (s - \min_s)/(\max_s - \min_s)$

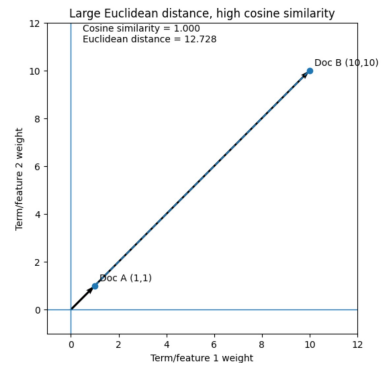
FOSTER SCHOOL OF BUSINESS

8

8

Math Expressions for Similarity Measures

- Assuming that Q and D are the sets of terms associated with a Query and a Document:
  - Simple matching $|Q \cap D|$
  - Dice's Coefficient $2\frac{|Q \cap D|}{|Q|+|D|}$
  - Jaccard Coefficient $\frac{|Q \cap D|}{|Q \cup D|}$
  - Cosine Coefficient $\frac{|Q \cap D|}{|Q|^{1/2} \times |D|^{1/2}}$
  - Overlap Coefficient $\frac{|Q \cap D|}{\min(|Q|,|D|)}$



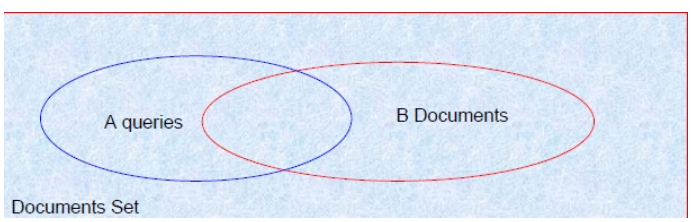
FOSTER SCHOOL OF BUSINESS

9

9

Overlap Coefficient

- Easier for vector manipulations



$$sim(q,d_j) = O(A,B) = \frac{|A \cap B|}{\min(|A|,|B|)}$$

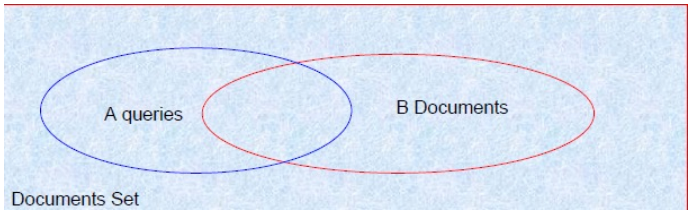
$$\cong \frac{\sum_{k=1}^{n} w_{kq} w_{kj}}{\min(\sum_{k=1}^{n} w_{kq}^2, \sum_{k=1}^{n} w_{kj}^2)}$$

FOSTER SCHOOL OF BUSINESS

10

10

Jaccard Coefficient



$$sim(q,d_j) = J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

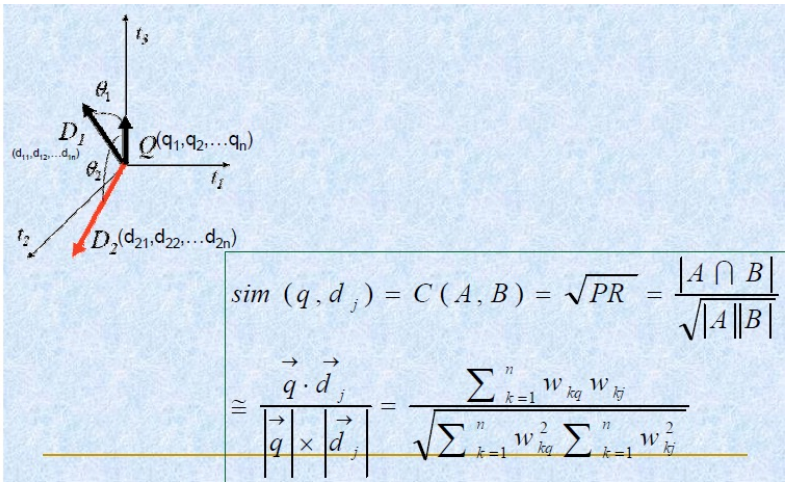
$$\cong \frac{\sum_{k=1}^{n} w_{kq} w_{kj}}{\sum_{k=1}^{n} w_{kq}^2 + \sum_{k=1}^{n} w_{kj}^2 - \sum_{k=1}^{n} w_{kq} w_{kj}}$$

FOSTER SCHOOL OF BUSINESS

11

11

Cosine Coefficient



$$sim(q,d_j) = C(A,B) = \sqrt{PR} = \frac{|A \cap B|}{\sqrt{|A||B|}}$$

$$\cong \frac{\vec{q} \cdot \vec{d}_j}{|\vec{q}| \times |\vec{d}_j|} = \frac{\sum_{k=1}^{n} w_{kq} w_{kj}}{\sqrt{\sum_{k=1}^{n} w_{kq}^2 \sum_{k=1}^{n} w_{kj}^2}}$$

FOSTER SCHOOL OF BUSINESS

12

12

When to Use Each Measure

- **Dice's Coefficient**: Use when you want to measure overlap but want to weight shared items a bit more than Jaccard, often in short texts or n-gram matching (e.g., in near-duplicate detection). It tends to give higher scores than Jaccard for the same overlap.
- **Jaccard Coefficient**: Use when you want a clean "intersection over union" notion of similarity on sets (unique tokens), especially when you care about distinct terms rather than how often they occur. It penalizes extra, non-overlapping terms more than Dice.
- **Cosine Coefficient**: Use when you represent documents as vectors with weights and want similarity that's robust to document length. This is the default for bag-of-words counts, TF-IDF, topic vectors, embeddings. It compares orientation, not magnitude, meaning two docs can be similar even if one is much longer, as long as term distribution is similar.
- **Overlap Coefficient**: Use when you care about containment: is the smaller document (or its phrases) largely contained in the larger? It equals 1.0 if the smaller set is a subset of the larger, even if the larger has lots of extra content. One use case is plagiarism or quote detection where one doc may include the other plus extra.

FOSTER SCHOOL OF BUSINESS

13

13

Practice

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import *
from sklearn.metrics.pairwise import *

vectorizer5 = CountVectorizer()
vectorizer5.fit(c3)
print(vectorizer5.vocabulary_)
v_c3 = vectorizer5.transform(c3).toarray()
print(v_c3)

cosine_similarity([v_c3[0]],[v_c3[1]])
jaccard_score(v_c3[0],v_c3[1])
```

FOSTER SCHOOL OF BUSINESS

14

14

Problem with Similarity

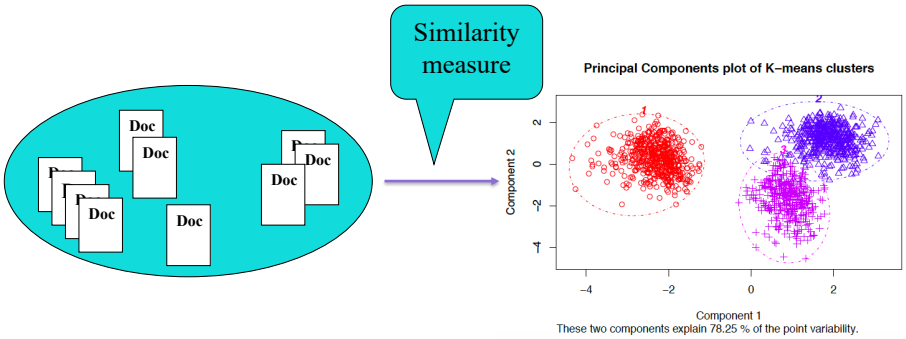
- Handling one document each time
- What if we want to handle all of the documents at once?

15

15

Clustering

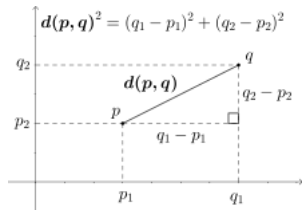
- **Given**: Similarity measures between vectors (documents)
- **Find**: Several clusters of vectors (documents) that are similar to each other



16

16

Euclidean Distance



$$dis(q, d_j) = d_E(q, d_j) = \sqrt{\sum_{k=1}^{n} (w_{kq} - w_{kj})^2}$$

17

17

Clustering Techniques

- Partitioning Methods
- Hierarchical Methods

18

18

Partitioning Algorithms

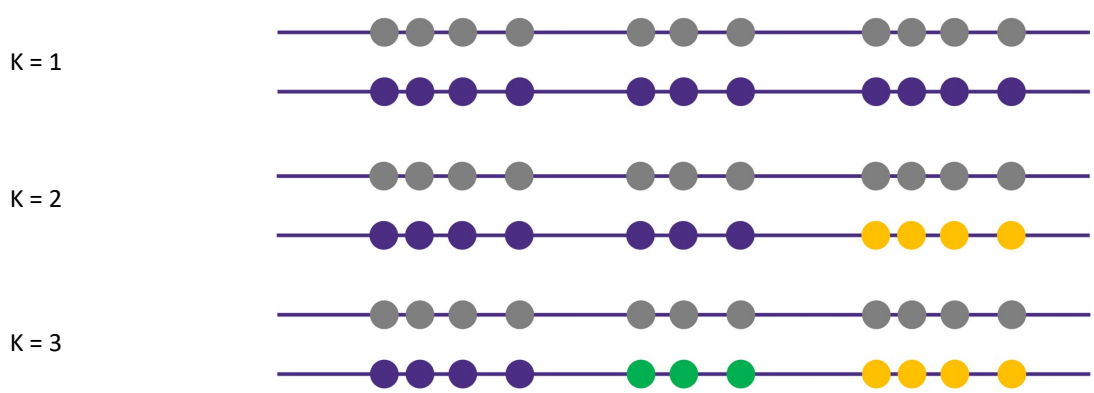
- **Partitioning method:** Construct a partition of ***n*** documents into a set of ***k*** clusters
- **Given:** a set of documents and the number ***k***
- **Find:** a partition of ***k clusters*** that optimizes the chosen partitioning criterion
  - **Global optimal**: exhaustively enumerate all partitions
  - Heuristic methods: **K-Means** algorithm
  - **K-Means**: Each cluster is represented by the center of the cluster

FOSTER SCHOOL OF BUSINESS

19

19

K-Means Clustering



FOSTER SCHOOL OF BUSINESS

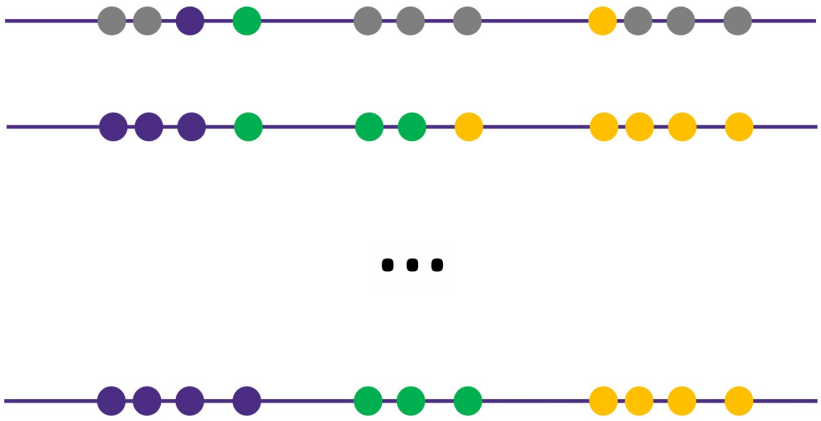
20

20

K-Means Clustering

K = 3

- Select random 3 dots
- For each non-selected dot, measure distance to the selected 3 dots, and assign it to the closest one.
- Calculate the mean (centroid) of each cluster;
- Re-measure the distance between each dot and the 3 new centroids, and reassign the clusters;
- Compute the total variation
- Try a new set of random 3 initial dots

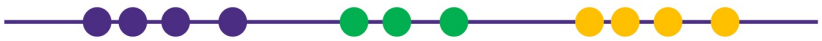


21

21

How Does the Algorithm Know the Best Clustering for a Given K?

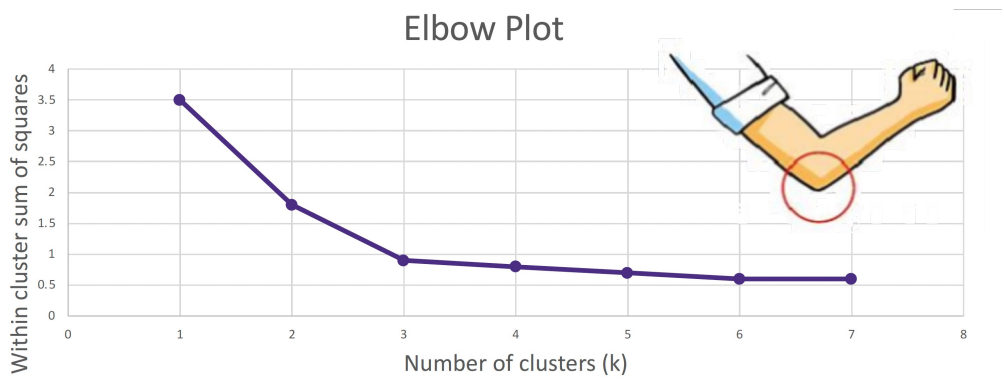
1. Full convergence
2. Minimize the sum of the squared distance between centroid and each member of the cluster (Within Cluster Sum of Squares)
3. Max iterations reached



22

22

How to Select K Using the Elbow Method?



23

23

Practice

```
c4 = [d0,d1,d2,d3,d4,d5,d6,d7,d8,d9]

vectorizer6 = TfidfVectorizer(stop_words='english')
X = vectorizer6.fit_transform(c4)

true_k = 4
model1 = KMeans(n_clusters=true_k, random_state=12, n_init=10)
model1.fit(X)
model1.cluster_centers_
model1.transform(X)
model1clusters = model1.labels_.tolist()
print(model1clusters)

order_centroids = model1.cluster_centers_.argsort()[:, ::-1]
terms = vectorizer6.get_feature_names_out()
for c in range(0, 4):
    print(f'Cluster {c}:')
    for ind in order_centroids[c, :4]:
        print(terms[ind])
```

24

24

Hierarchical Clustering

- Agglomerative:
  - Start with each document being a single cluster.
  - In each step, merge the two closest clusters.
  - Eventually all document belong to the same cluster.
  - Objective function: minimizes the sum of squared distances within all clusters.
- Divisive:
  - Start with all documents belonging to the same cluster.
  - Eventually each node forms a cluster on its own.
  - In each step, separate the two farthest potential clusters

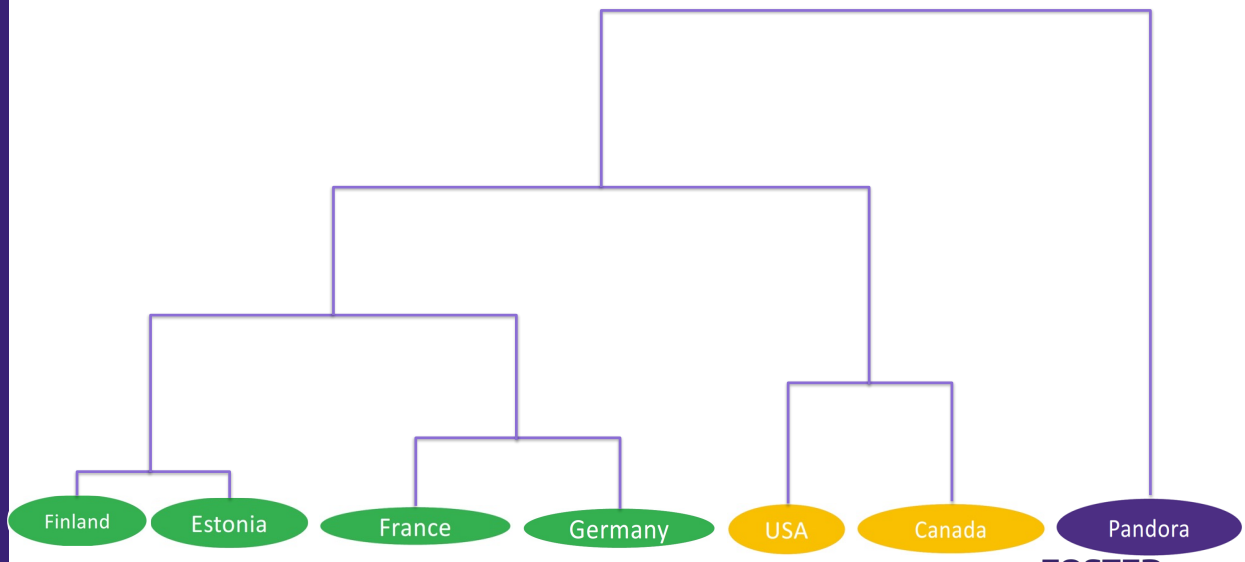
- Note: does not require the number of clusters k in advance

FOSTER SCHOOL OF BUSINESS

25

25

Flat vs. Hierarchical Clustering



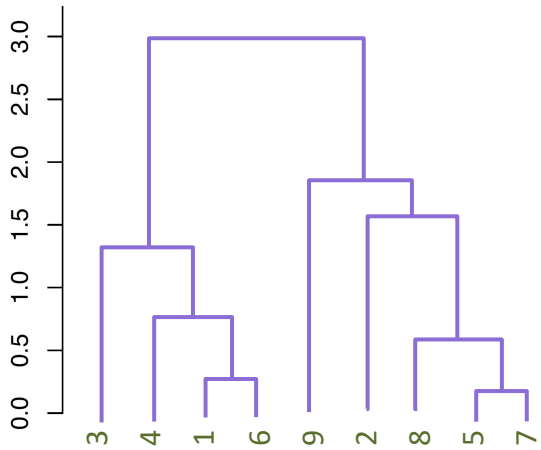
FOSTER SCHOOL OF BUSINESS

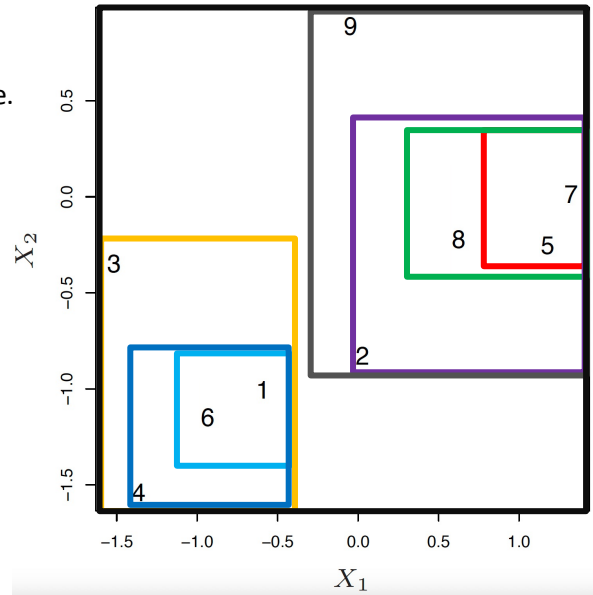
26

26

How Does the Dendrogram Work?

- A dendrogram basically keeps track of all the fuses.
- Height of fusing indicates how similar the points are.





27

27

Practice

```
## Hierarchical clustering

from sklearn.cluster import AgglomerativeClustering

model2 = AgglomerativeClustering(n_clusters=4,
                  metric='euclidean', linkage=complete’)

model2.fit_predict(X.toarray())
print(model2.labels_)
```

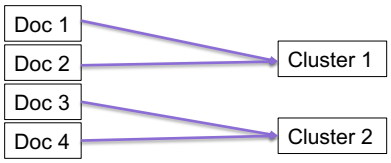
FOSTER SCHOOL OF BUSINESS

28

28

Limitations of Clustering

- Fundamental assumption: each doc belongs to one cluster, many to one relationship



- What if a doc might belong to multiple clusters?
  - E.g., can a movie (review) be related to both action and horror?
- Question: When a doc appears to belong to multiple clusters, how do we model this?

FOSTER SCHOOL OF BUSINESS

29

29

Topic Modeling:
Latent Dirichlet Allocation (LDA)

FOSTER SCHOOL OF BUSINESS

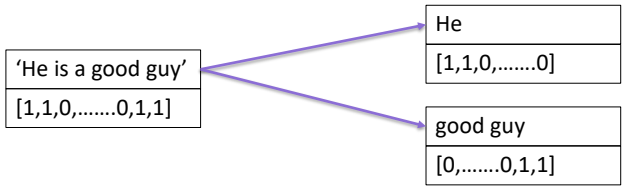
30

30

Topic Modeling

- **One data point** (vector) is part of **multiple (many) topics**
- **One topic** corresponds to specific **values** in the feature vector

- **One document** is part of **multiple (many) topics**
- **One** topic consists of **multiple** regularly co-occurring words (example topics: "sports", "medicine", "movies", "finance")



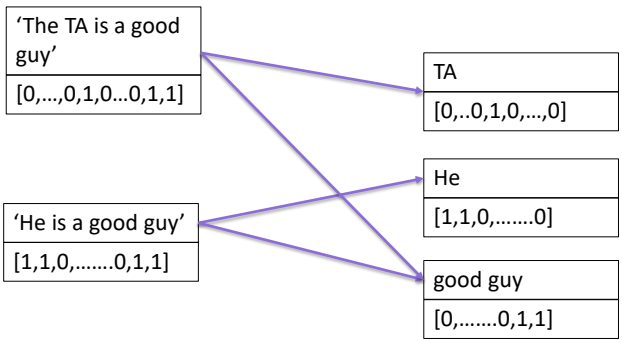
FOSTER SCHOOL OF BUSINESS

31

31

Topic Modeling

- A doc can be better described by many topics
  - Many to many relationship
- How to do that?
  - Multiple methods, but LDA is the most popular one



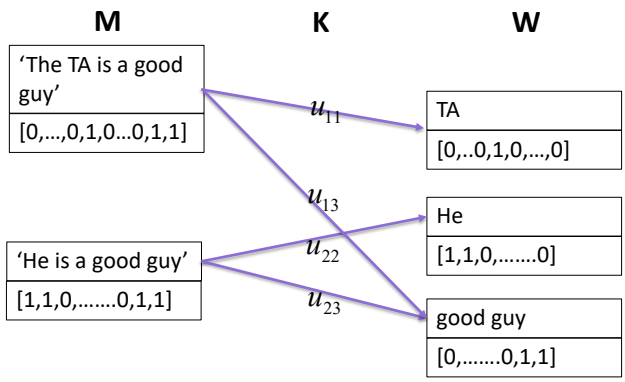
FOSTER SCHOOL OF BUSINESS

32

32

Latent Dirichlet Allocation (LDA)

- Ref: Latent Dirichlet Allocation by Blei, Ng and Jordan
  - Documents are produced from a mixture of topics
  - Topics generate words based on a probability distribution (generative probability model)



FOSTER SCHOOL OF BUSINESS

33

33

Matrix Factorization

- General idea: perform a **low-rank approximation** of **document-term matrix**
- **M** = **KW**
  - **M**: document-term matrix
  - **K**: P(topic *t* | document *d*): each document's topic weight distribution
  - **W**: P(word *w*| topic *t*): each topic's words distribution

	W1	W2	W3	Wn
D1	0	2	1	3
D2	1	4	0	0
D3	0	2	3	1
Dn	1	1	3	0

=

	K1	K2	K3	K
D1	1	0	0	1
D2	1	1	0	0
D3	1	0	0	1
Dn	1	0	1	0

✖

	W1	W2	W3	Wm
K1	0	1	1	1
K2	1	1	1	0
K3	1	0	0	1
K	1	1	0	0

FOSTER SCHOOL OF BUSINESS

34

34

Practice: Use sklearn

```
from sklearn.feature_extraction.text import CountVectorizer
vectorizer7 = CountVectorizer(stop_words='english')
X2 = vectorizer7.fit_transform(c4)
terms = vectorizer7.get_feature_names_out()

from sklearn.decomposition import LatentDirichletAllocation
lda = LatentDirichletAllocation(n_components=4, random_state=12).fit(X2)

for topic_idx, topic in enumerate(lda.components_):
    print(f'Topic {topic_idx}:')
    print(" ".join([terms[i] for i in topic.argsort()[:-4-1:-1]]))

lda.transform(X2[0])
```

FOSTER SCHOOL OF BUSINESS

35

35

Practice: Use gensim

```
import gensim
dictionary = gensim.corpora.Dictionary(processed_c4)
bow_c4 = [dictionary.doc2bow(doc) for doc in processed_c4]
print(bow_c4[0])

lda_model = gensim.models.LdaModel(bow_c4, num_topics=4, id2word=dictionary)

for idx, topic in lda_model.print_topics(-1, 4):
    print('Topic: {} \n Words: {}'.format(idx, topic))

lda_model[bow_c4[0]]
```

FOSTER SCHOOL OF BUSINESS

36

36

Sentiment Analysis

FOSTER SCHOOL OF BUSINESS

37

37

Classification in a Descriptive Way: Sentiment Analysis

- If we have a classifier already
  - E.g., a dictionary that includes a sentiment score for each word
- Use the classifier to calculate a sentiment analysis score for documents
- He is good -> [0,0,1,0,0,0] -> score = 1/3 -> **positive**
- He is bad and crazy -> [0,0,0,-1,0,-0.5] ->score = -1.5/5 -> **negative**

FOSTER SCHOOL OF BUSINESS

38

38

Sentiment Analysis Example 1: Positivity and Negativity

```
from nltk.sentiment.vader import SentimentIntensityAnalyzer
import pandas as pd
nltk.download('vader_lexicon')

sentences = [”They are smart, cute, and funny.",  # positive sentence example
    “They are smart, cute, and funny!", # punctuation emphasis handled correctly (sentiment intensity
adjusted)
    “They are very smart, cute, and funny.",# booster words handled correctly (sentiment intensity adjusted)
    "They are VERY SMART, cute, and FUNNY.",  # emphasis for ALLCAPS handled
    “They are VERY SMART, cute, and FUNNY!!!",  # combination of signals - VADER appropriately adjusts
intensity
    “They are VERY SMART, really handsome, and INCREDIBLY FUNNY!!!",  # booster words & punctuation make this
close to ceiling for score
    "The book was good.",  # positive sentence
    "The book was kind of good.",  # qualified positive sentence is handled correctly (intensity adjusted)
    "The plot was good, but the characters are uncompelling and the dialog is not great.",  # mixed negation
sentence
    "A really bad, horrible book.",  # negative sentence with booster words
    "At least it isn't a horrible book.",  # negated negative sentence with contraction
    ":) and :D",  # emoticons handled
    "",  # an empty string is correctly handled
    "Today sux",  # negative slang handled
    "Today sux!",  # negative slang with punctuation emphasis handled
    "Today SUX!",  # negative slang with capitalization emphasis
    "Today kinda sux! But I'll get by, lol"  # mixed sentiment example with slang and contrastive conjunction on
"but"
    ]
```

39

39

Sentiment Analysis Example 2: Positivity and Negativity

```
## Apply existing classifier

sid = SentimentIntensityAnalyzer()

scores = []
for sentence in sentences:
    score = {'sentence': sentence}
    score.update(sid.polarity_scores(sentence))
    scores.append(score)

df_scores = pd.DataFrame(scores)
df_scores
```

40

40

More Advanced Models

- RNN: Recurrent Neural Network
- LSTM: Long Short-Term Memory

- Transformer models
  - BERT: Bidirectional Encoder Representations from Transformers
  - ChatGPT: Generative Pretrained Transformer

FOSTER SCHOOL OF BUSINESS